Motivation for Transactions

Grouping user actions (reads & writes) into *transactions* helps with two goals:

1. **Recovery and Durability**: Keeping the DB data consistent and durable in the face of crashes, aborts, system shutdowns, etc.

2. **Concurrency**: Achieving better performance by parallelizing TXNs *without* creating anomalies

Motivation -- Recovery & Durability

1. **Recovery and durability** of user data is essential for reliable database (and other data science systems)

- The database may experience crashes (e.g. power outages, etc.)
- Individual TXNs may be aborted (e.g. by the user)

Idea: Make sure that TXNs are either **durably stored in full**, **or not at all**; keep log to be able to “roll-back” TXNs

Protection against crashes / aborts

Client 1:

```
INSERT INTO SmallProduct(name, price)
SELECT pname, price
FROM Product
WHERE price <= 0.99
```

Crash / abort!

```
DELETE Product
WHERE price <=0.99
```

What goes wrong?

Protection against crashes / aborts

```
Client 1:
        START TRANSACTION
        INSERT INTO SmallProduct(name, price)
                SELECT pname, price
                FROM Product
                WHERE price <= 0.99

        DELETE Product
                WHERE price <=0.99
        COMMIT OR ROLLBACK
```

Now we'd be fine! We'll see how / why this lecture

Motivation -- Concurrent execution

2. **Concurrent** execution of user programs is essential for good database performance.

- Disk accesses may be frequent and slow: optimize for throughput (# of TXNs), trade for latency (time for any one TXN)
- Users should still be able to execute TXNs as if in isolation and such that consistency is maintained

Idea: Have the database handle running several user TXNs concurrently, in order to keep throughput high

Multiple users: single statements

```
Client 1: UPDATE Product
                    SET Price = Price – 1.99
                    WHERE pname = 'Gizmo'

Client 2:   UPDATE Product
                    SET Price = Price*0.5
                    WHERE pname = 'Gizmo'
```

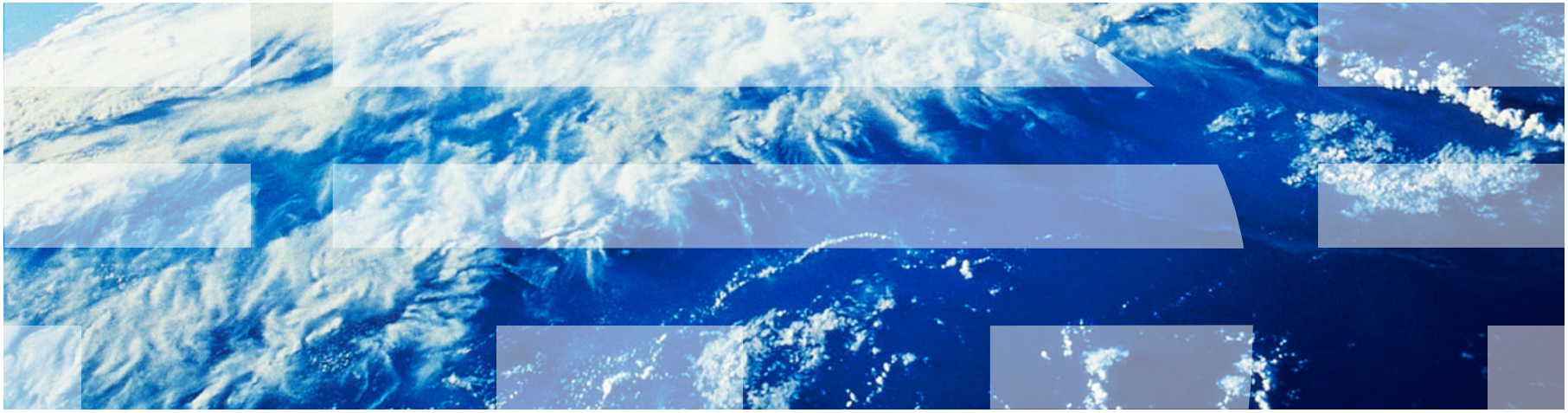
Two managers attempt to discount products *concurrently*-
What could go wrong?

Multiple users: single statements

```
Client 1: START TRANSACTION
                                    UPDATE Product
                                    SET Price = Price – 1.99
                                    WHERE pname = ‘Gizmo’
                          COMMIT

Client 2: START TRANSACTION
                                    UPDATE Product
                                    SET Price = Price*0.5
                                    WHERE pname=‘Gizmo’
              COMMIT
```

ACID
Atomicity, Consistency, Isolation, Durability



Transaction Properties: ACID

- Atomic
  - State shows either all the effects of txn, or none of them
- Consistent
  - Txn moves from a state where integrity holds, to another where integrity holds
- Isolated
  - Effect of txns is the same as txns running one after another
- Durable
  - Once a txn has committed, its effects remain in the database

ACID continues to be a source of great debate!
(in fact, by default, most databases don't provide it…)

ACID: Atomicity

- Txn's activities are atomic: all or nothing
  - Intuitively: in the real world, a transaction is something that would either occur *completely* or *not at all*
- Two possible outcomes for a txn
  - It *commits*: all the changes are made
  - It *aborts*: no changes are made

A<u>C</u>ID: <u>C</u>onsistency

- The tables must always satisfy user-specified *integrity constraints*
  - *Examples:*
    - Account number is unique (primary key constraint)
    - Stock amount can't be negative
    - Sum of *debits* and of *credits* is 0

- How consistency is achieved:
  - Programmer writes a TXN to go from one consistent state to another consistent state
  - *System* makes sure that the TXN is atomic
  - → Assuming system maintaining atomicity, this is often the user's responsibility

FINANCE

Europe

Irish bank glitch let customers pull out large sums of ‘free money’—sparking huge run on ATMs

By **Chloe Taylor**

August 16, 2023, 10:08 AM ET

Add us on

A woman at a Bank of Ireland ATM in Clifden, April 2021. Thanks to a glitch, some Bank of Ireland customers recently found themselves able to withdraw hundreds of euros they didn’t own from their accounts, prompting long lines at ATMs.

ARTUR WIDAK—NURPHOTO/GETTY IMAGES

https://fortune.com/europe/2023/08/16/bank-of-ireland-free-money-glitch-lines-at-atms-police-garda-intervention-dublin/

ACID: Isolation

- A transaction executes concurrently with other transactions
- **Isolation**: the effect is as if each transaction executes in *isolation* of the others.
  - E.g. Transaction A not be able to observe changes from another concurrent transaction B during the run

ACID: Durability

- The effect of a TXN must continue to exist (*"persist"*) after the TXN
  - And after the whole program has terminated
  - And even if there are power failures, crashes, etc.
  - And etc…
- Means: Write data to disk
  - And in data center settings: replicate data, backup, etc.

Challenges for ACID properties

- In spite of power failures (i.e., in spite of loss of memory)
- Users may abort the program: need to “rollback changes”
  - Need to *log* what happened
- Many users executing concurrently

And all this with… Scalability and/or Performance!!